

Practical 1

```
import numpy as np

# Spreading codes
c1 = [1, 1, 1, 1]
c2 = [1, -1, 1, -1]
c3 = [1, 1, -1, -1]
c4 = [1, -1, -1, 1]

rc = []

print("Enter the data bits:")

d1 = int(input("Enter D1: "))
d2 = int(input("Enter D2: "))
d3 = int(input("Enter D3: "))
d4 = int(input("Enter D4: "))

# Multiply data with spreading codes
r1 = np.multiply(c1, d1)
r2 = np.multiply(c2, d2)
r3 = np.multiply(c3, d3)
r4 = np.multiply(c4, d4)

# Resultant channel signal
resultant_channel = r1 + r2 + r3 + r4
print("Resultant Channel:", resultant_channel)

# Select receiver code
Channel = int(input("Enter the station to listen for (C1=1, C2=2, C3=3, C4=4): "))

if Channel == 1:
    rc = c1
elif Channel == 2:
    rc = c2
elif Channel == 3:
    rc = c3
elif Channel == 4:
    rc = c4
else:
    print("Invalid channel selected")
    exit()

# Inner product calculation
inner_product = np.multiply(resultant_channel, rc)
print("Inner Product:", inner_product)

res1 = sum(inner_product)

data = res1 / len(inner_product)

print("Data bit that was sent:", data)
```

OUTPUT

```
print("Data bit that was sent:", data)

✓ ... Enter the data bits:
Enter D1: 1
Enter D2: -1
Enter D3: 1
Enter D4: -1
Resultant Channel: [0 4 0 0]
Enter the station to listen for (C1=1, C2=2, C3=3, C4=4): 2
Inner Product: [ 0 -4  0  0]
Data bit that was sent: -1.0
```

Practical 4

```
clear
```

```
N = 10^6; % number of bits or symbols
```

```
rand('seed',100); % initializing the rand() function
```

```
randn('seed',200); % initializing the randn() function
```

```
% Transmitter
```

```
ip = rand(1,N) > 0.5; % generating 0,1 with equal probability
```

```
s = 2*ip - 1; % BPSK modulation 0 -> -1; 1 -> 1
```

```
n = 1/sqrt(2) * [randn(1,N) + j*randn(1,N)]; % white gaussian noise, 0dB variance
```

```
Eb_N0_dB = [-3:10]; % multiple Eb/N0 values
```

```
for ii = 1:length(Eb_N0_dB)
```

```
    % Noise addition
```

```
    y = s + 10^(-Eb_N0_dB(ii)/20) * n; % additive white gaussian noise
```

```
    % receiver - hard decision decoding
```

```
    ipHat = real(y) > 0;
```

```
    % counting the errors
```

```
    nErr(ii) = size(find([ip - ipHat]),2);
```

```
end
```

```
simBer = nErr / N; % simulated ber
```

```
theoryBer = 0.5 * erfc(sqrt(10.^(Eb_N0_dB/10))); % theoretical ber
```

```
% plot
```

```
close all
```

```
figure
```

```
semilogy(Eb_N0_dB, theoryBer, 'b.-');
```

```
hold on
```

```
semilogy(Eb_N0_dB, simBer, 'mx-');
```

```
axis([-3 10 10^-5 0.5])
```

```
grid on
```

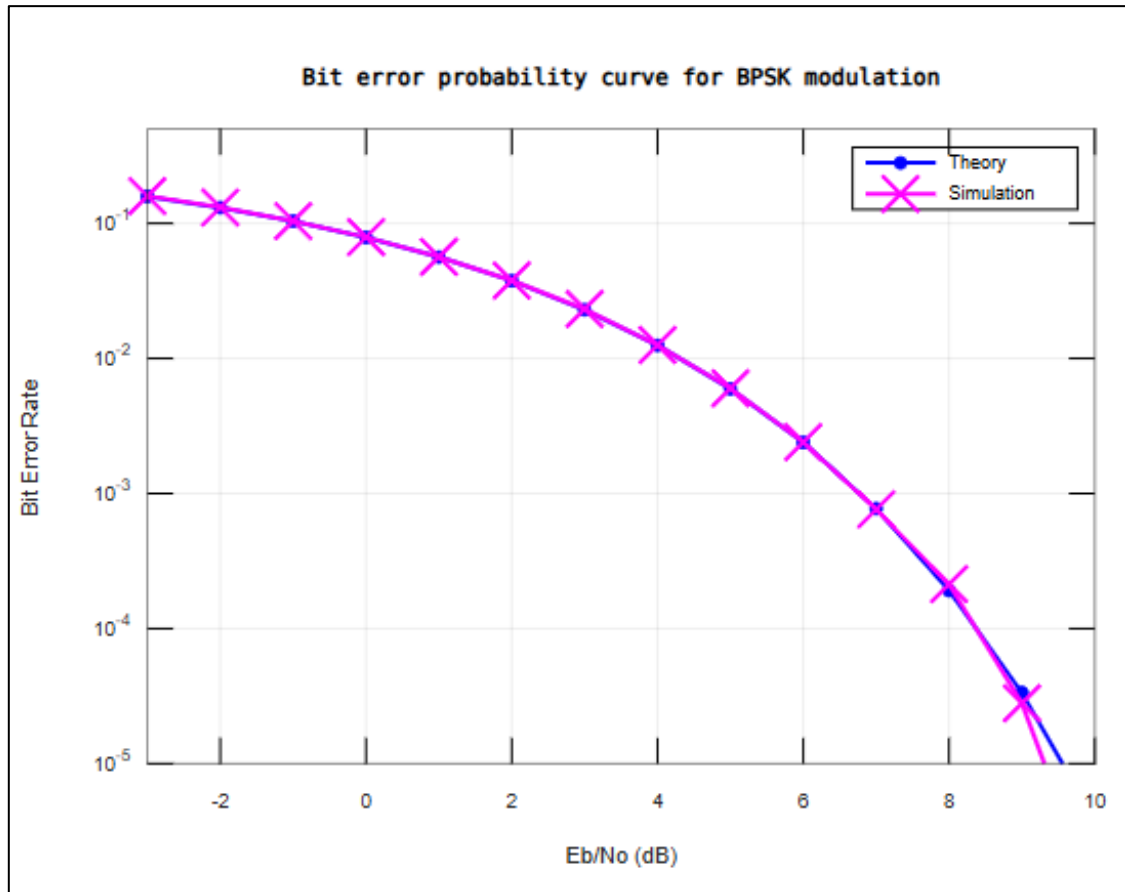
```
legend('theory', 'simulation');
```

```
xlabel('Eb/No, dB');
```

```
ylabel('Bit Error Rate');
```

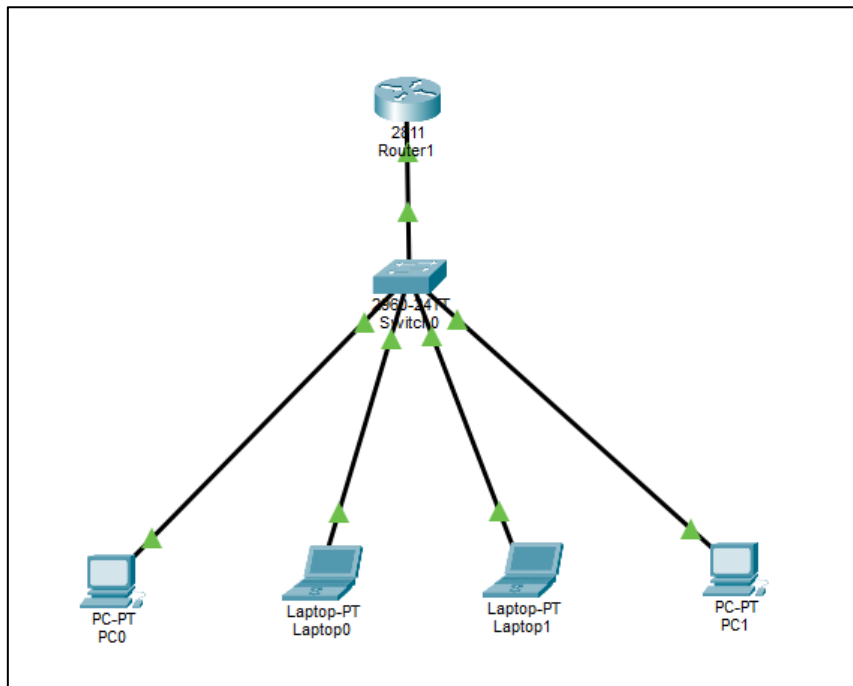
```
title('Bit error probability curve for BPSK modulation');
```

OUTPUT



Practical 5

Network consisting of router, switch and end devices



Configuring the router as a DHCP Server

```
--- System Configuration Dialog ---

Would you like to enter the initial configuration dialog? [yes/no]: no

Press RETURN to get started!

Router>en
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#int fa0/0
Router(config-if)#ip add 192.168.1.1 255.255.255.0
Router(config-if)#no sh

Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up

Router(config-if)#exit
Router(config)#ip dhcp pool pool1
Router(dhcp-config)#network 192.168.1.0 255.255.255.0
Router(dhcp-config)#default-router 192.168.1.1
Router(dhcp-config)#
```

Assigning IP Address using DHCP

Laptop0

Physical Config **Desktop** Programming Attributes

IP Configuration X

Interface FastEthernet0

IP Configuration

☒ DHCP ☐ Static DHCP request successful.

IPv4 Address 192.168.1.3

Subnet Mask 255.255.255.0

Default Gateway 192.168.1.1

DNS Server 0.0.0.0

IPv6 Configuration

☐ Automatic ☒ Static

IPv6 Address /

Link Local Address FE80::20B:BEFF:FE0C:4DC9

Default Gateway

DNS Server

802.1X

☐ Use 802.1X Security

Authentication MD5

Username

Password

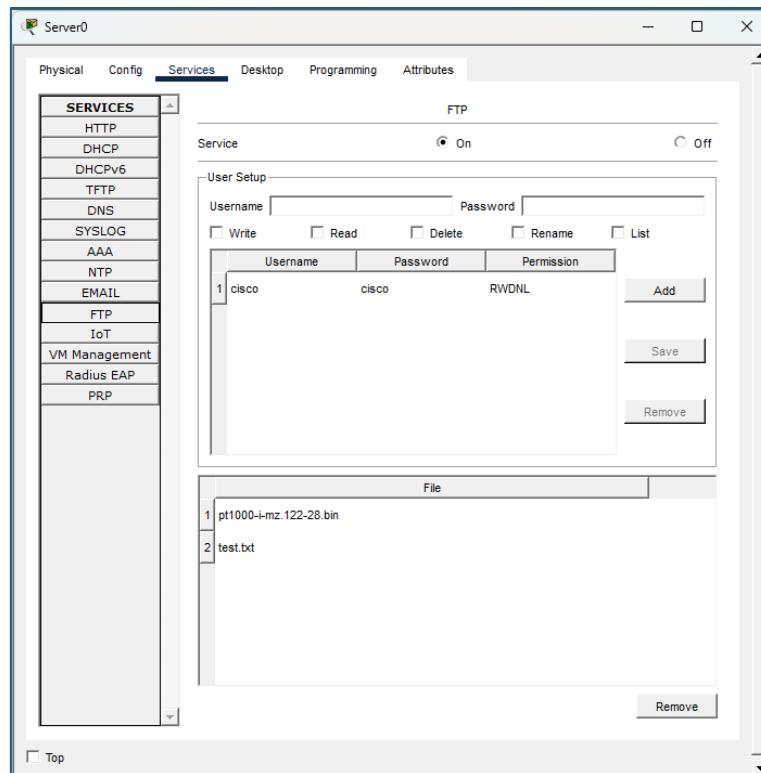
☐ Top

DHCP Server Practical:

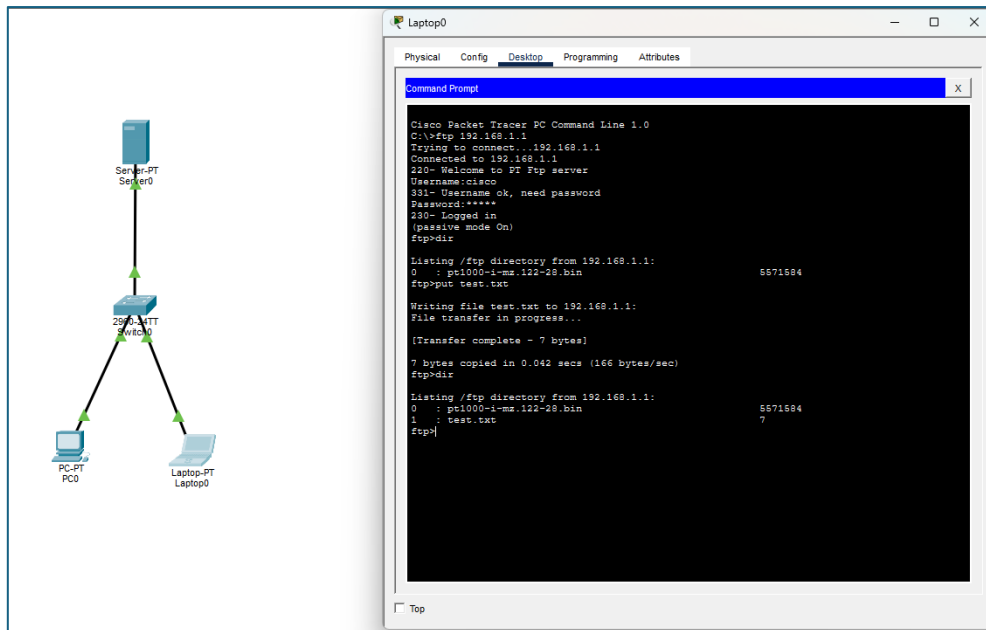
```
en
conf t
int fa0/0
ip add 192.168.1.1 255.255.255.0
no sh
exit
ip dhcp pool pool1
network 192.168.1.0 255.255.255.0
default-router 192.168.1.1
```

Practical 8

Configuring FTP on the server:



Sending files from Laptop to server using FTP



```
FTP-
ftp 192.168.1.1
username:cisco
password:cisco
dir
put test.txt
dir
```

Sending files from Laptop to server using FTP using python

```
|— server.py
|— client.py
|— abc.txt
```

```
-----
|                               SERVER.PY                               |
-----
```

```
import socket
```

```
HOST = "localhost"
```

```
PORT = 5000
```

```
FORMAT = "utf-8"
```

```
# Create TCP socket
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
# Bind IP and port
```

```
server.bind((HOST, PORT))
```

```
# Wait for client
```

```
server.listen(1)
```



```
print("Waiting for connection...")

# Accept client
conn, addr = server.accept()

print("Connected by:", addr)

# Receive filename
filename = conn.recv(1024).decode(FORMAT)

print("Receiving file:", filename)

# Send acknowledgement
conn.send("Filename received".encode(FORMAT))

# Open file to save data
file = open(filename, "wb")

# Receive file data continuously
while True:

    data = conn.recv(1024)

    if not data:
        break

    file.write(data)

print("File received successfully")

# Close everything
file.close()
conn.close()
server.close()
```

	CLIENT.PY	
--	-----------	--

```
import socket

HOST = "localhost"
PORT = 5000
FORMAT = "utf-8"

FILE_NAME = "abc.txt"
```

```
# Create TCP socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Connect to server
client.connect((HOST, PORT))

# Send filename
client.send(FILE_NAME.encode(FORMAT))

# Receive acknowledgement
msg = client.recv(1024).decode(FORMAT)

print(msg)

# Open file
file = open(FILE_NAME, "rb")

# Send file data continuously
while True:

    data = file.read(1024)

    if not data:
        break

    client.send(data)

print("File sent successfully")

# Close everything
file.close()
client.close()
```

Name	Date modified	Type	Size
Today			
abc.txt	5/10/2026 1:42 PM	Text Document	0 KB
client.py	5/10/2026 1:35 PM	Python File	1 KB
server.py	5/10/2026 1:34 PM	Python File	1 KB

C:\Windows\System32\cmd.e X + v

Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

E:\Downloads\python ftp>python server.py
Waiting for connection...
Connected by: ('127.0.0.1', 61546)
Receiving file: abc.txt
File received successfully

E:\Downloads\python ftp>

C:\Windows\System32\cmd.e X + v

Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

E:\Downloads\python ftp>python client.py
Filename received
File sent successfully

E:\Downloads\python ftp>